Project Report: Stage 1 - Railway Track Fault Detection

Submission Date: 3 November 2025

Team Members:

GAJENDRA (2301ME18)

MOHIT MEENA (2301ME34)

MOHIT SAHU (2301ME35)

PRIYANSHU PRIYAM (2301ME41)

SAI KUMAR MALAVATH (2303ME14)

---

1. Dataset Selection

As per the project guidelines[1], we selected a publicly available image dataset from Kaggle.

- **Dataset:** Railway Track Fault Detection
- **Source:** Kaggle (https://www.kaggle.com/datasets/gpiosenka/railway-track-fault-detection-resized-224-x-224)
- **Description:** The dataset contains images of railway tracks. The images are categorized into two classes defective and non-defective.
- We have a total sample of 300 training images and 22 test and 62 validation images.



Defective



Non-Defective

---

2. Problem Definition

The objective of this project is to perform a **binary image classification task**. The goal is to train a model that can accurately classify an image of a railway track as either **"defective"** or **"non-defective"**. This automation task is crucial for improving railway safety and maintenance efficiency.

---

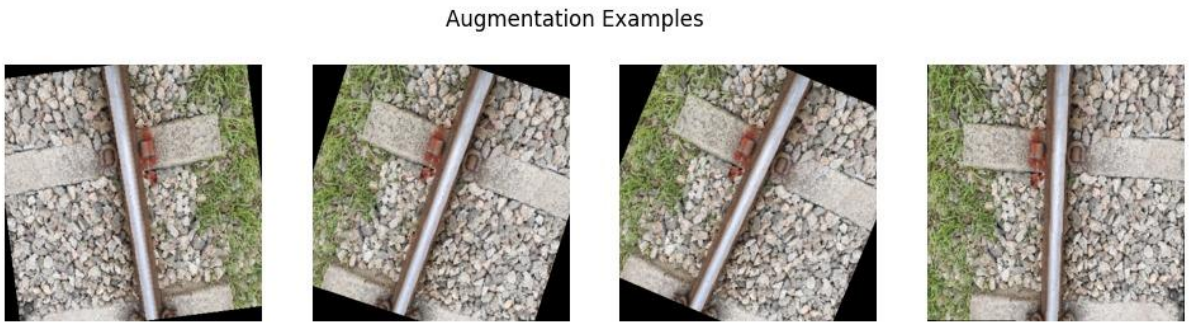
3. Model Implementation

We implemented a deep learning model using the **PyTorch and Torchvision** frameworks.

3.1. Data Preprocessing

To prepare the data for the neural network, we performed the following steps:

- **Data Augmentation:** Rotated, flipped the images.
- **Resizing:** All images were resized to a uniform dimension (384x384 pixels) to ensure compatibility with the model's input layer.



3.2. Model Architecture

We implemented two different architectures.

3.2.1 Simple CNN

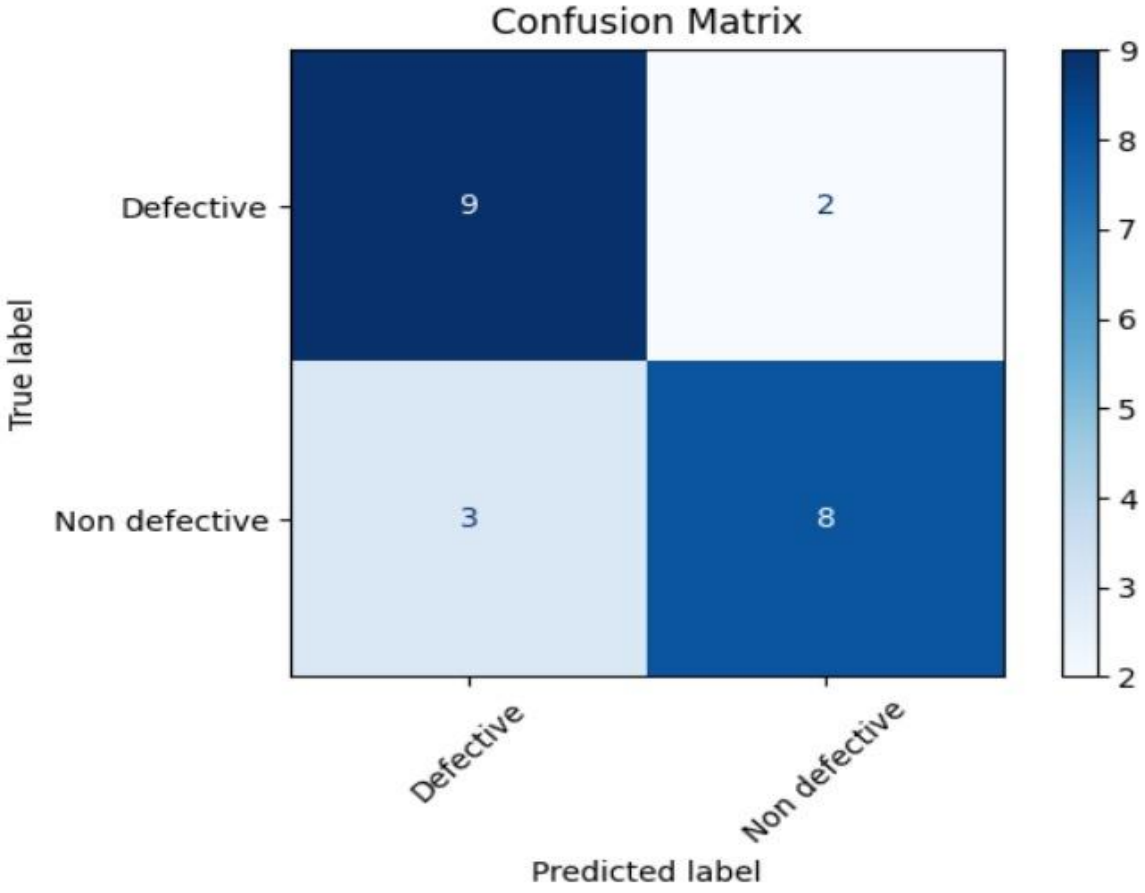
We implemented a **Convolutional Neural Network (CNN),** and optimised its architecture using optuna hyper parameter search consisting of 50 trials.

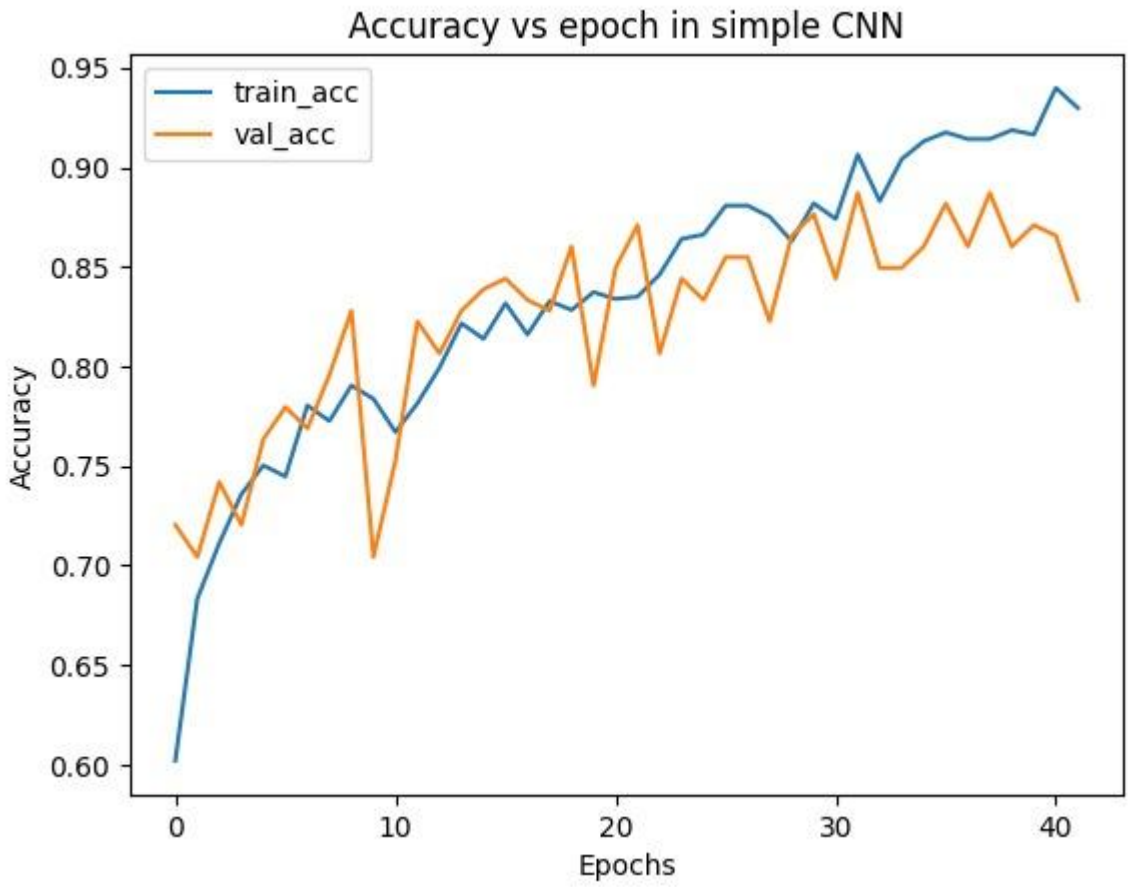
- Two convolutional layers with relu Activation and Max Pulling
- Deep Neural Network with 1 hidden layer consisting of 256 neurons.
- Optimiser: Adam
- Dropout Rate: 0.2
- Loss Function: Cross Entropy Loss.
- Epoch: 30 with early stopping of patience 10.

Scores

Metric	Score
Best Validation Accuracy	**0.8871**
Test Loss	**0.8980**
Accuracy	**0.7121**
Precision	**0.7123**
Recall	**0.7121**
F1-Score	**0.7121**

Confusion Matrix





3.2.2 Transfer Learning

Here we used pretrain model to extract features from images and then train a model on those images for classification.

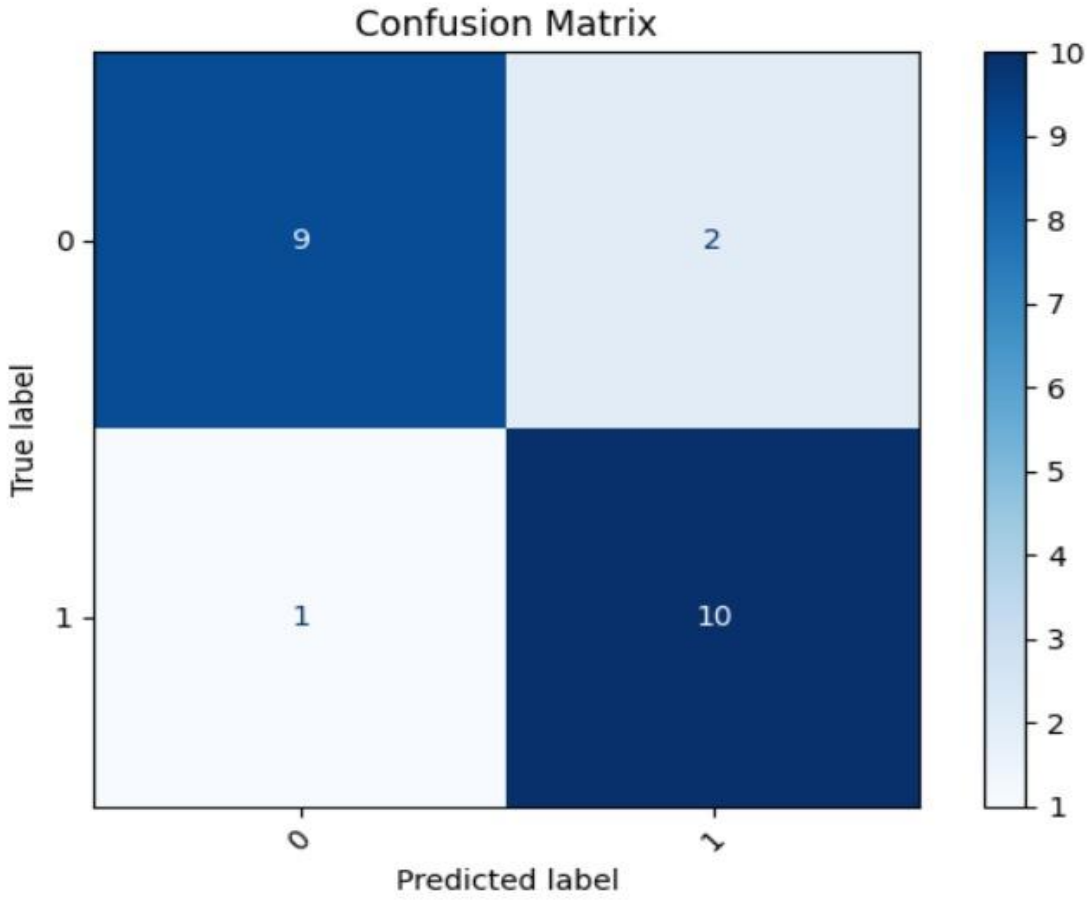
- **Pretrain Model:** torchvision.models.ViT_B_16_Weights.IMAGENET1K_SWAG_E2E_V1
- **Classification Model:** One hidden layer with 256 Neurons and relu activation.
- Dropout Rate: 0.5
- Optimiser: RMSprop
- Loss Function: Cross Entropy Loss.
- Epoch: 30 with early stopping of patience 10.

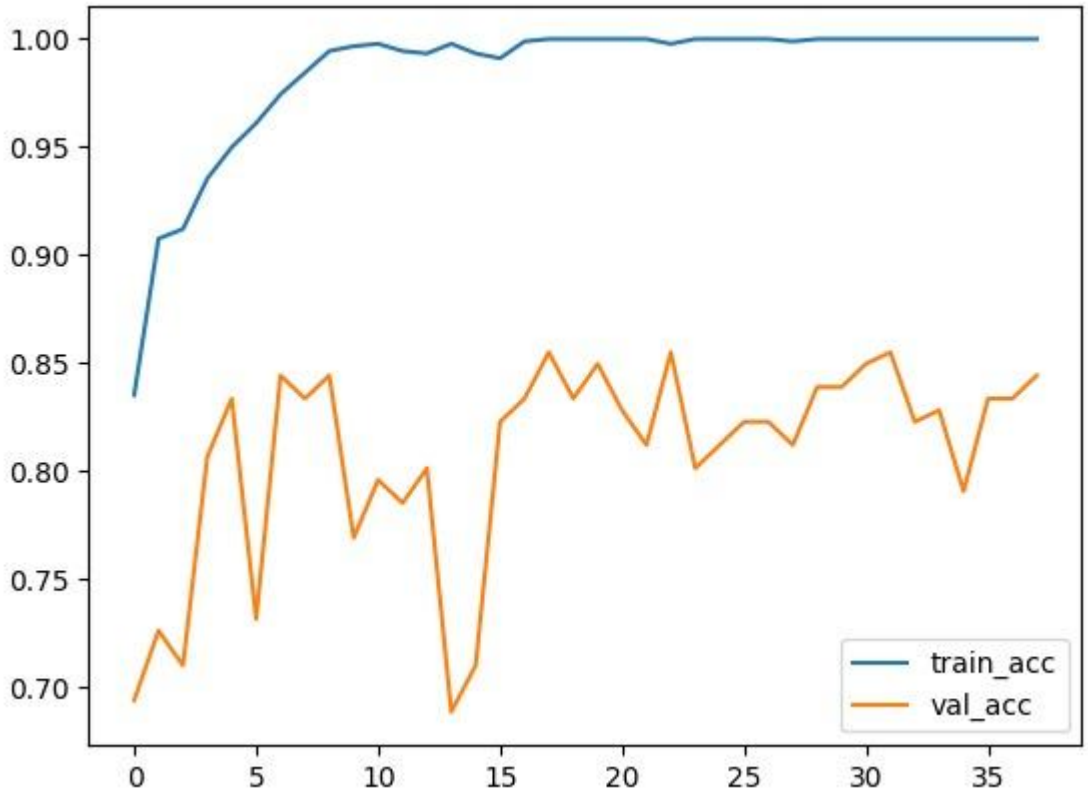
SCORES:

Metric	Score
Best Validation Accuracy	0.8656

Test Loss	0.5256
Accuracy	0.8636
Precision	0.8929
Recall	0.8636
F1-Score	0.8611

Confusion Matrix:




1.00
0.95
0.90
0.85
0.80
0.75
0.70
0
5
10
15
20
25
30
35
train_acc
val_acc